



《AI大模型Ragent项目》——RAG知识库管理宏观设计

来自： 拿个offer-开源&项目实战



马丁

2026年03月24日 22:59

前面的理论篇里，咱们把 RAG 的核心环节都讲完了：数据分块、元数据管理、向量化、向量数据库、检索策略、生成策略。理论上，你已经知道怎么把文档切成 chunk、怎么生成向量、怎么检索、怎么生成答案。

但假设现在老板让你做一个电商智能客服系统，用 RAG 来回答用户关于商品、退换货、售后的问题。你会发现理论和实战之间有一道鸿沟。运营同学上传的文档格式五花八门（PDF、Word、Excel、Markdown），有些 PDF 还是扫描版的，怎么解析？文档处理要好几分钟，运营同学等不了，怎么异步处理？处理到一半失败了，怎么重试？商品手册更新了，旧的 chunk 怎么办？电商平台有多个品类（手机、家电、服装），每个品类要独立的知识库，怎么隔离？

这些工程问题，理论篇都没讲。这篇文档就是要填补这个鸿沟，从宏观层面讲清楚 RAG 知识库系统的整体设计。

从理论到实战的鸿沟

1. 理论篇讲了什么

回顾一下前面的理论篇，咱们讲了 RAG 的核心环节：

- 数据分块**：把长文档切成适合检索的小段，讲了固定大小、重叠、递归、语义等分块策略
- 元数据管理**：给每个 chunk 贴上标签，记录来源、位置等信息，方便引用和过滤
- 向量化**：把文本 chunk 转成数字向量，让计算机能做语义级别的相似度检索
- 向量数据库**：用专门的数据库存储向量并做 ANN 近似最近邻搜索，讲了 HNSW 索引等算法
- 检索策略**：纯向量检索、BM25 关键词检索、混合检索、重排序，提升检索质量
- 生成策略**：Prompt 工程、幻觉抑制、引用对齐，让大模型基于检索结果生成高质量答案

这些是 RAG 的理论基础，但它们都是单点技术，讲的是某个环节怎么做。真要做一个完整的知识库系统，你会发现还有很多工程问题没解决。

2. 实战会遇到什么问题

假设你要做一个电商智能客服系统，运营同学可以上传商品手册、退换货政策、售后服务指南等文档，用户提问时系统检索相关内容并生成答案。听起来很简单，但实际动手会遇到一堆问题：

- 文档格式多样**：运营同学上传的文档格式五花八门，PDF、Word、Excel、Markdown、纯文本，甚至还有扫描版 PDF。每种格式的解析方式不一样，PDF 可能有复杂的排版和表格，Excel 是表格数据等。怎么统一处理？
- 处理耗时长**：一个 100 页的商品手册 PDF，解析要几秒，分块要几秒，向量化要调几十上百次 API，整个流程下来可能要几分钟。运营同学上传文档后不能一直等着，必须异步处理。怎么设计异步流程？
- 失败要重试**：文档解析可能失败（文件损坏、格式不支持），向量化 API 可能超时，数据库写入可能失败。失败了怎么重试？重试几次？怎么记录失败原因？
- 文档更新**：运营同学上传了 iPhone 16 Pro 产品手册 v1.0，过了一个月苹果更新了保修政策，又上传了 v2.0。旧版本的 chunk 怎么处理？直接删掉还是标记为失效？删除的时候新向量还没生成，用户问答搜不到怎么办？
- 多知识库隔离**：电商平台有多个品类，手机数码、家用电器、服装鞋包，每个品类要独立的知识库，互不干扰。用户问手机问题时，不应该检索到服装的文档。怎么设计数据库表结构实现隔离？
- 成本控制**：向量化要调 Embedding API，按 Token 计费。1000 个文档，每个 100 个 chunk，共 10 万个 chunk，调用成本多少？怎么优化？

这些问题，理论篇都没讲，但实际做系统必须解决。

3. 本篇要解决什么

这篇文档是实战系列的开篇，重点讲宏观设计，不深入代码细节。你会了解到：

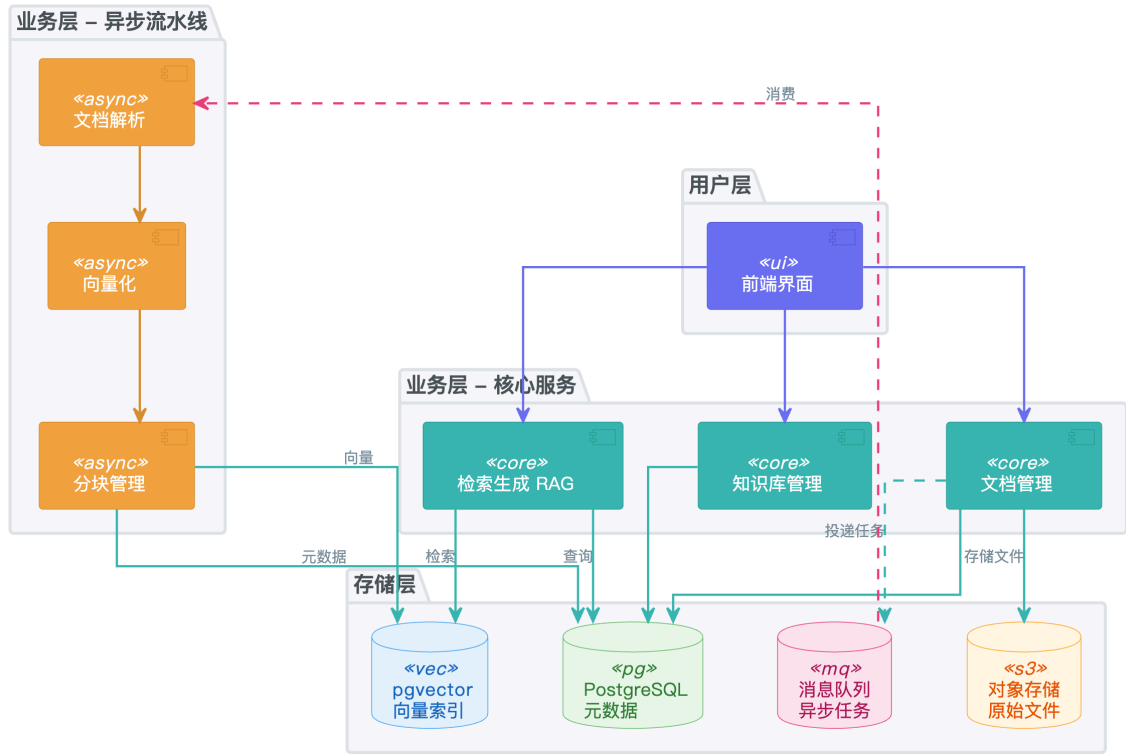
- 三层架构**：知识库系统的整体架构，用户层、业务层、存储层各自的职责
- 核心概念**：知识库、文档、分块三个核心概念，以及它们之间的关系
- 数据库设计**：三张核心表的字段设计和关系，为什么这么设计
- 处理流程**：文档从上传到入库的完整流程，异步处理、状态流转、异常重试
- 关键难点**：文档解析、分块质量、向量化成本、数据一致性等真实的坑

后续的实践篇会深入每个环节的具体实现，比如文档解析实战、分块策略实战、Pipeline 架构实战等。

知识库系统的三层架构

1. 架构总览

企业级知识库系统可以分成三层：用户层、业务层、存储层。这个分层主要是为了职责清晰、方便扩展。



这张图展示了三层架构的全貌。用户层是前端界面，用户在这里创建知识库、上传文档、提问。业务层是核心逻辑，包括知识库管理、文档管理、分块管理、文档解析、向量化、检索生成。存储层是数据持久化，包括 PostgreSQL 存元数据、pgvector 存向量、对象存储存原始文件、消息队列做异步任务。

2. 用户层：前端交互

用户层就是前端界面，用户能做的事情包括：

- 创建知识库：**给知识库起个名字，选择 Embedding 模型（比如 bge-m3、qwen-embedding-8b）
- 上传文档：**支持本地上传、URL 抓取同步等多种方式
- 查看文档：**看文档列表，查看处理状态（处理中、成功、失败），查看失败原因
- 向量分块：**对文档执行分块逻辑，分为直接分块和数据通道（ETL）分块策略
- 查看分块：**看某个文档被切成了多少个 chunk，每个 chunk 的内容和元数据

用户层的设计要简单直观，用户不需要懂 RAG 的原理，只需要知道上传文档、提问、看答案。

3. 业务层：核心逻辑

业务层是系统的核心，包括五大模块：

- 知识库管理：**创建、更新、删除知识库，配置 Embedding 模型。每个知识库是独立的，有自己的 Embedding 模型配置和向量 Collection，不同知识库的数据互不干扰。
- 文档管理：**上传文档、查看文档列表、查看文档详情、删除文档。文档上传后会触发异步处理流程，文档管理模块负责把任务发到消息队列。
- 分块管理：**管理 chunk 的增删改查，支持按知识库、按文档查询 chunk。分块管理模块还负责 chunk 的去重，避免重复内容多次入库。
- 文档解析：**从原始文件中提取纯文本。支持 PDF、Word、Excel、Markdown 等多种格式，使用 Apache Tika 做统一解析。扫描版 PDF 需要 OCR 识别。
- 向量化：**调用 Embedding API 把文本 chunk 转成向量。支持批量调用（一次 10~20 个 chunk），降低 API 调用次数和成本。

文档管理模块只负责接收上传请求和发任务，不关心文档怎么解析。文档解析模块只负责解析，不关心解析后的文本怎么分块。这样每个模块都很简单，方便测试和维护。

4. 存储层：数据持久化

存储层包括四个组件：

PostgreSQL：存储知识库、文档、分块的元数据。为什么用 PostgreSQL？因为它是关系型数据库，支持事务、复杂查询，适合存储结构化的元数据。

pgvector：PostgreSQL 的向量扩展，支持向量存储和 ANN 检索。为什么不用 Milvus？因为 pgvector 和 PostgreSQL 在同一个数据库里，元数据和向量在一起，事务一致性更好，运维更简单，成本更低。pgvector 支持 HNSW 索引，性能够用（百万级向量）。

还有就是，Milvus 部署成本比较高，一个是复杂度一个是本地内存占用等；肯定是希望大家学起来顺畅，综合考虑选择 pgvector。

对象存储：存储用户上传的原始文件。可以用 S3、阿里云 OSS、本地文件系统。咱们使用的是 RustFS，不过依赖的是 S3 协议，理论上很多对象存储可以无缝迁移。

消息队列：异步处理文档解析、分块、向量化任务。可以用 RocketMQ、Redis Stream。为什么要异步？因为文档处理耗时长（几十几分钟），不能阻塞用户请求。消息队列支持失败重试、并发控制、任务优先级。

为了保障大家快速学习项目，项目默认使用 Redis Stream，后续会出教程如何使用 RocketMQ，以及如何使用 RocketMQ 的事务消息平衡数据库和三方组件间的事务性。

这四个组件各有分工，PostgreSQL 存元数据，pgvector 存向量，对象存储存文件，消息队列做异步任务。

核心概念模型

知识库系统有三个核心概念：知识库、文档、分块。它们是一对多的关系，一个知识库包含多个文档，一个文档包含多个分块。理解这三个概念，就理解了知识库系统的数据模型。

1. 知识库（Knowledge Base）

1.1 知识库是什么

知识库是文档的容器，类似文件夹。你可以创建多个知识库，比如手机数码知识库、家电知识库、服装知识库。每个知识库是独立的，有自己的配置，互不干扰。

为什么要有知识库这个概念？直接把所有文档放在一起不行吗？不行，因为：

品类隔离：不同品类的文档要分开，用户问手机问题时，不应该检索到服装的文档，否则答非所问

模型隔离：不同知识库可能用不同的 Embedding 模型，中文商品用中文模型，英文商品用英文模型

配置独立：不同知识库可以有不同的分块策略、检索参数，方便针对性优化

1.2 核心属性

一个知识库有这些核心属性：

id：知识库唯一标识，VARCHAR(20)，用雪花算法生成的分布式 ID

name：知识库名称，比如手机数码知识库、家电知识库

embedding_model：使用的 Embedding 模型标识，比如 bge-m3、qwen-embedding-8b

collection_name：向量库的 Collection 名称，唯一标识，用于在 pgvector 中隔离不同知识库的向量数据

created_by、updated_by：创建者和修改者 ID

create_time、update_time：创建时间、更新时间

其中 embedding_model 是关键字段。为什么要记录 Embedding 模型？因为不同模型生成的向量维度不一样，不能混用。如果你用 bge-m3（1024 维）生成了向量，后来换成 qwen-embedding-8b（4096 维），旧的向量就不能用了，必须重新生成。

collection_name 也很关键，它是向量数据在 pgvector 中的逻辑分割标识。每个知识库对应一个 Collection，不同知识库的向量数据通过 Collection 隔离，检索时只在指定 Collection 内搜索。

1.3 为什么需要知识库概念

打个比方，知识库就像电商平台的品类分区。京东有手机数码、家用电器、服装鞋包等分区，每个分区的商品是独立管理的，搜索也是分区搜索的。知识库系统也一样，不同知识库的文档是独立管理的，检索也是分知识库检索的。

如果没有知识库概念，所有文档放一起，会有这些问题：

用户问 iPhone 16 Pro 的保修期，系统可能检索到冰箱的保修政策，答非所问

不同品类的文档混在一起，检索噪音大，相关性低

想用不同的 Embedding 模型处理不同类型的文档，做不到

有了知识库概念，这些问题都解决了。用户提问时，通过咱们意图识别树定位意图，然后意图绑定知识库，然后只在这个知识库里检索。不同知识库可以用不同的 Embedding 模型，互不影响。

2. 知识库文档 (Knowledge Document)

2.1 文档是什么

文档是知识库的基本单元，一个文档对应一个上传的文件。运营同学上传一个 PDF 商品手册，就是一个文档。上传一个 Markdown 格式的退换货政策，也是一个文档。

文档有生命周期：上传 → 处理中 → 成功 / 失败。处理过程中，系统会自动完成解析、分块、向量化、入库等环节。

2.2 核心属性

Ragent 项目中，一个文档有这些核心属性：

- id：文档唯一标识
- kbId：所属知识库 ID
- docName：文档名称，比如“iPhone 16 Pro 产品手册.pdf”
- fileUrl：文件存储路径，上传后保存到对象存储的路径
- fileType：文件类型，比如 pdf、docx、md、txt
- fileSize：文件大小，单位字节
- processMode：处理模式，chunk（直接分块）或 pipeline（走 Pipeline 流水线）
- status：处理状态，pending（待处理）、running（处理中）、success（成功）、failed（失败）
- sourceType：来源类型，file（本地上传）或 url（URL 抓取）
- sourceLocation：来源地址，URL 抓取时记录原始 URL
- chunkStrategy：分块策略，比如 recursive（递归分块）、fixed_size（固定大小）
- chunkConfig：分块配置，JSONB 类型，存储 chunkSize、overlapSize 等参数
- pipelineId：Pipeline ID，processMode=pipeline 时关联的 Pipeline 定义
- chunkCount：分块数量，冗余字段，方便统计
- enabled：是否启用，1 启用，0 禁用
- scheduleEnabled：是否启用定时刷新
- scheduleCron：定时表达式，比如 0 0 2 * * ?（每天凌晨 2 点刷新）

这里有几个字段值得展开说说。

2.3 两种处理模式：chunk vs pipeline

processMode 字段决定了文档的处理方式，Ragent 支持两种模式：

- chunk 模式**：直接分块。系统按照文档配置的 chunkStrategy 和 chunkConfig 做解析、分块、向量化、入库。这是最常用的模式，适合大部分文档。
- pipeline 模式**：走 Pipeline 流水线。文档关联一个预定义的 Pipeline（通过 pipelineId），Pipeline 由多个节点组成（获取 → 解析 → 清洗 → 分块 → 增强 → 索引），每个节点可以单独配置。这种模式更灵活，适合复杂的文档处理场景，比如需要特殊清洗规则、需要对 chunk 做增强（加摘要、加关键词）等。

简单场景用 chunk 模式就够了，复杂场景再上 Pipeline 模式。

Pipeline 本质就是 ETL，只不过 T（Transform）阶段从字段映射进化成了 AI 语义处理。ETL 本质上说的是：

- Extract 提取（从各种源拿到原始数据）
- Transform 转换（清洗 + 分块 + 增强）
- Load 加载（写入向量数据库 + 元数据存储）

大家知道后续讲 Pipeline 和 ETL 说的是一件事情就好。

2.4 分块配置：文档级别可定制

chunkStrategy 和 chunkConfig 这两个字段让每个文档可以有自己的分块配置，而不是知识库统一一套。

为什么需要文档级别的分块配置？因为同一个知识库里，文档类型可能差别很大。比如手机数码知识库里，有 200 页的产品手册（长文档，适合大 chunk），也有一页纸的常见问题 FAQ（短文档，适合小 chunk）。如果所有文档用同一套分块参数，效果不会太好。

chunkConfig 是 JSONB 类型，存储分块参数：

```
{  
  "chunkSize": 512,  
  "overlapSize": 128  
}
```

2.5 定时刷新：文档自动更新

`scheduleEnabled` 和 `scheduleCron` 这两个字段支持文档定时刷新。

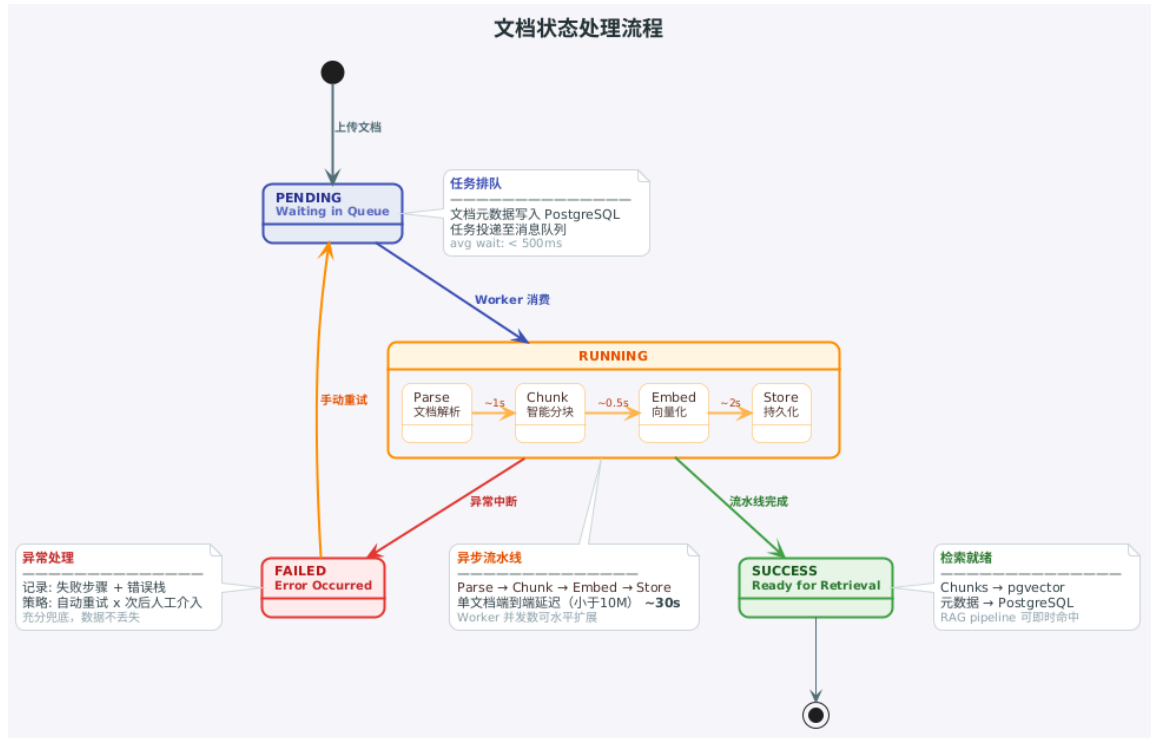
什么场景需要定时刷新？比如运营同学通过 URL 导入了一个在线文档，这个文档每周更新一次。手动每周重新上传太麻烦，配置一个定时任务，系统自动去抓取最新内容，重新解析、分块、入库。

还有一种可能性，很多人改了在线文档，没有意识上传到向量知识库，导致回答不是最新的！！！别问我是怎么知道的，说多了都是泪。

`scheduleCron` 用的是标准的 Cron 表达式，比如 `0 0 2 * * ?` 表示每天凌晨 2 点执行。

2.6 文档状态流转

文档的状态流转很简洁：`pending` → `running` → `success` / `failed`。



上传文档后状态是 `pending`，用户触发分块时通过 CAS 操作把状态改成 `running`（仅当当前不是 `running` 时才能改，防止重复触发），异步处理完成后改成 `success`，失败则改成 `failed`。

和前面提示词里设计的六个状态（`pending` → `parsing` → `chunking` → `indexing` → `success` / `failed`）不同，Ragent 实际只用了四个状态。为什么不拆得更细？因为解析、分块、向量化、入库是在同一个异步任务里顺序执行的，中间的状态切换意义不大——用户关心的是整体结果（成功还是失败），不关心跑到哪一步了。如果失败了，通过分块日志表（`t_knowledge_document_chunk_log`）就能定位到具体环节和耗时。

这种设计更简洁，状态流转更清晰，代码也更好维护。

3. 知识库分块 (Knowledge Chunk)

3.1 分块是什么

分块是文档的片段，是检索的基本单元。一个 100 页的 PDF 可能被切成 200 个 chunk，每个 chunk 大约 500 字。用户提问时，系统检索的是 chunk，不是整个文档。

为什么要分块？因为大模型的上下文窗口有限，不能把整个文档都塞进去。而且整个文档里可能只有一小段和用户问题相关，把整个文档都给模型会引入大量噪音。分块后，系统可以精准检索到相关的那几段，生成的答案质量更高。

3.2 核心属性

Ragent 项目中，一个分块有这些核心属性：

- `id`: 分块唯一标识
- `kbId`: 所属知识库 ID
- `docId`: 所属文档 ID
- `chunkIndex`: 在文档中的序号，从 0 开始
- `content`: 分块文本内容
- `contentHash`: 内容哈希，用于去重
- `charCount`: 字符数
- `tokenCount`: Token 数，用于成本估算
- `enabled`: 是否启用，1 表示启用，0 表示禁用

你可能会问：向量存在哪里？元数据存在哪里？

3.3 向量和元数据的存储：分离设计

Ragent 的 chunk 表里**没有** embedding 向量字段和 metadata 元数据字段。向量和元数据是单独存储在向量数据库中的，不和 chunk 的业务数据混在一起。

为什么要这么设计？

职责分离：t_knowledge_chunk 表负责存储 chunk 的业务数据（文本内容、所属文档、序号等），向量数据库负责存储向量和做相似度检索。两个存储各司其职，互不干扰。

灵活性：向量数据库可以独立选型和替换。Ragent 目前用 pgvector，但如果以后数据量大了需要换成 Milvus 或 Qdrant，只需要改向量存储层的代码，chunk 表不用动。

性能：向量字段很大（一个 4096 维的 float32 向量占 16KB），如果放在 chunk 表里，会严重影响普通查询的性能（比如分页查询 chunk 列表）。分开存储后，chunk 表的查询不受向量字段拖累。

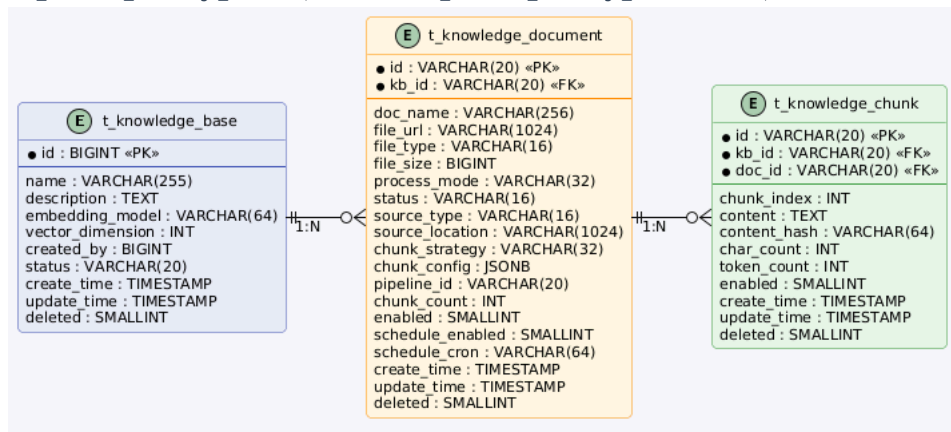
具体来说，向量化完成后，向量数据会写入 pgvector 的 Collection 中，每条记录包含 chunk ID、内容、向量、以及需要的元数据字段。

数据库表设计

理解了三个核心概念，接下来看数据库表怎么设计。知识库系统有三张核心表：t_knowledge_base、t_knowledge_document、t_knowledge_chunk，它们是一对多的关系。

1. 表结构总览

一个知识库包含多个文档，一个文档包含多个分块。t_knowledge_document 通过 kb_id 关联 t_knowledge_base，t_knowledge_chunk 通过 doc_id 关联 t_knowledge_document，同时也通过 kb_id 关联 t_knowledge_base（冗余字段，方便查询）。



后续知识库相关库表可能会有部分 SQL 变更，但是内核定义是不会变的。

2. t_knowledge_base（知识库表）

2.1 表结构定义

```
CREATE TABLE t_knowledge_base (
    id          VARCHAR(20) NOT NULL PRIMARY KEY,
    name        VARCHAR(128) NOT NULL,
    embedding_model VARCHAR(64) NOT NULL,
    collection_name VARCHAR(64) NOT NULL,
    created_by   VARCHAR(20) NOT NULL,
    updated_by   VARCHAR(20),
    create_time  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_time  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    deleted      SMALLINT NOT NULL DEFAULT 0,
    CONSTRAINT uk_collection_name UNIQUE (collection_name)
);
CREATE INDEX idx_kb_name ON t_knowledge_base (name);
COMMENT ON TABLE t_knowledge_base IS '知识库表';
```

2.2 设计要点

核心字段的含义在上文核心概念模型章节已经详细说明，这里补充几个表结构层面的设计考量：

- embedding_model 绑定在知识库级别，创建后不建议修改，否则已有向量全部失效
- collection_name 加了唯一约束，保证每个知识库在向量库中的数据隔离
- deleted 是软删除标记，删除知识库是高危操作，方便后续追溯

3. t_knowledge_document（文档表）

3.1 表结构定义

```
CREATE TABLE t_knowledge_document (  
  id          VARCHAR(20)      NOT NULL PRIMARY KEY,  
  kb_id       VARCHAR(20)      NOT NULL,  
  doc_name    VARCHAR(256)    NOT NULL,  
  enabled     SMALLINT        NOT NULL DEFAULT 1,  
  chunk_count INTEGER         DEFAULT 0,  
  file_url    VARCHAR(1024)    NOT NULL,  
  file_type   VARCHAR(16)     NOT NULL,  
  file_size   BIGINT,  
  process_mode VARCHAR(16)    DEFAULT 'chunk',  
  status      VARCHAR(16)     NOT NULL DEFAULT 'pending',  
  source_type  VARCHAR(16),  
  source_location VARCHAR(1024),  
  schedule_enabled SMALLINT,  
  schedule_cron VARCHAR(64),  
  chunk_strategy VARCHAR(32),  
  chunk_config JSONB,  
  pipeline_id  VARCHAR(20),  
  created_by   VARCHAR(20)    NOT NULL,  
  updated_by   VARCHAR(20),  
  create_time  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  update_time  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  deleted      SMALLINT       NOT NULL DEFAULT 0  
);  
CREATE INDEX idx_kb_id ON t_knowledge_document (kb_id);  
COMMENT ON TABLE t_knowledge_document IS '知识库文档表';
```

3.2 设计要点

字段含义详见上文知识库文档章节的核心属性和各子节说明，这里补充表结构层面的设计考量：

- chunk_count 是冗余字段，避免每次查文档列表都要去 t_knowledge_chunk 表 COUNT，提升查询性能
- chunk_config 用 JSONB 类型存储，灵活支持不同分块策略的参数，比如 {"chunkSize": 500, "overlapSize": 50}
- enabled 和 deleted 是两个维度的控制：enabled=0 表示暂时禁用（可随时启用），deleted=1 表示软删除（逻辑上已不存在）
- kb_id 上建了索引，因为按知识库查文档列表是最常见的查询

4. t_knowledge_chunk（分块表）

4.1 表结构定义

```
CREATE TABLE t_knowledge_chunk (  
  id          VARCHAR(20)      NOT NULL PRIMARY KEY,  
  kb_id       VARCHAR(20)      NOT NULL,  
  doc_id      VARCHAR(20)      NOT NULL,  
  chunk_index INTEGER         NOT NULL,  
  content     TEXT             NOT NULL,  
  content_hash VARCHAR(64),  
  char_count  INTEGER,  
  token_count INTEGER,  
  enabled     SMALLINT        NOT NULL DEFAULT 1,  
  created_by  VARCHAR(20)     NOT NULL,  
  updated_by  VARCHAR(20),  
  create_time TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  update_time TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  deleted     SMALLINT       NOT NULL DEFAULT 0  
);
```

```
);  
CREATE INDEX idx_doc_id ON t_knowledge_chunk (doc_id);  
COMMENT ON TABLE t_knowledge_chunk IS '知识库文档分块表';
```

4.2 设计要点

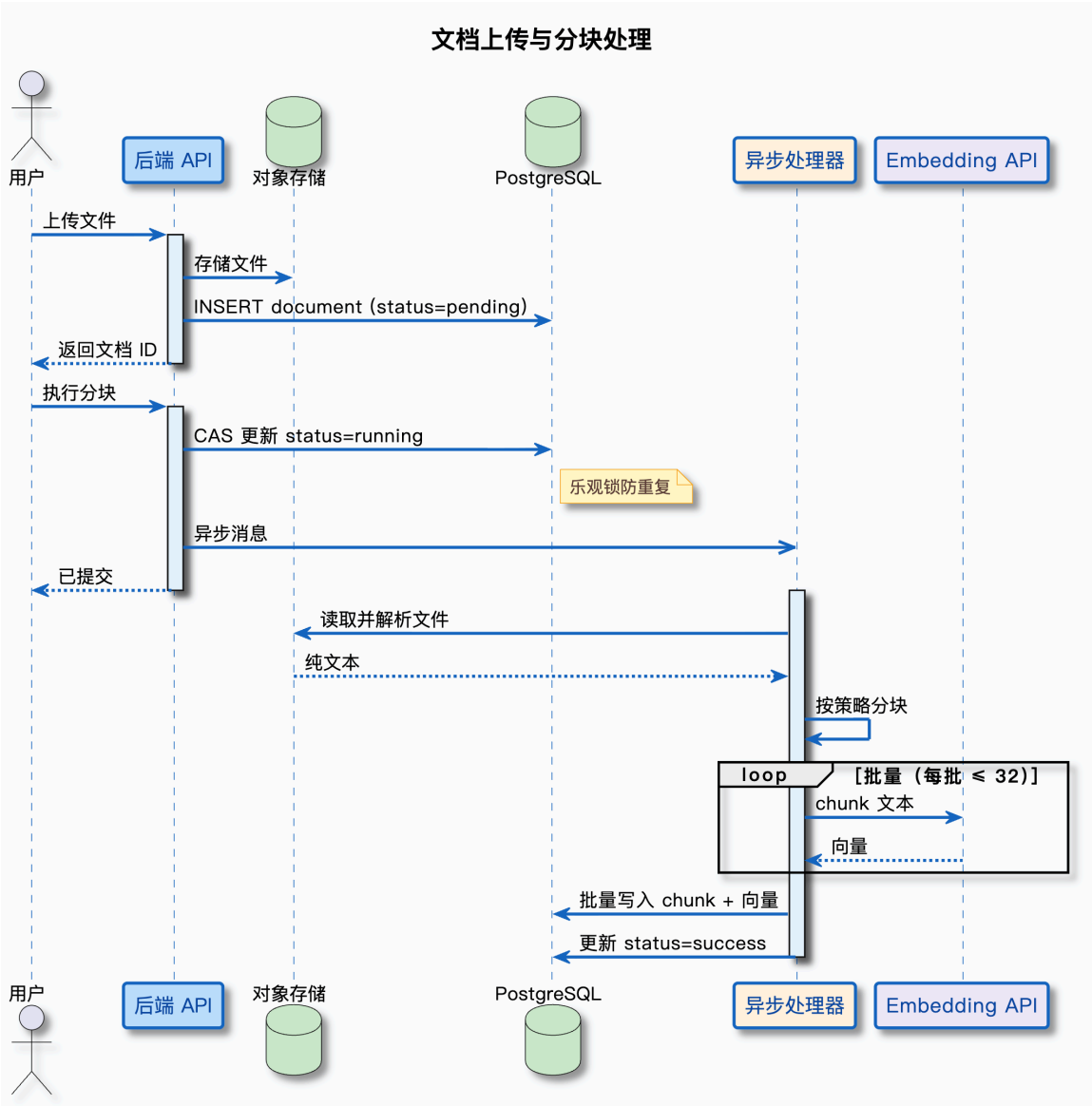
字段含义详见上文知识库分块章节的核心属性说明，这里补充表结构层面的设计考量：

- content 是 TEXT 类型，不限长度，实际长度由分块策略的 chunkSize 决定
- content_hash 用于去重，具体去重机制见下文分块去重章节
- kb_id 是冗余字段，chunk 本身通过 doc_id 关联文档、文档再关联知识库，但按知识库维度查询 chunk 很常见，冗余 kb_id 避免 JOIN
- doc_id 上建了索引，因为按文档查询 chunk 列表是最常见的查询
- 向量数据不在这张表里，分离存储在 pgvector 中，原因见上文分离设计章节

文档上传与处理流程

理解了数据库设计，接下来看文档从上传到入库的完整流程。Ragent 把这个流程拆成了两个独立的接口：先上传文档，再触发分块。分块处理是异步的，通过消息队列触发。

1. 完整流程图



这张图展示了 Ragent 的两阶段处理流程。上传和分块是两个独立的操作：用户先上传文档，文档状态变成 pending；然后在前端点击执行分块，后端通过 CAS（Compare-And-Swap）把状态改成 running，防止重复触发，然后发消息到队列。异步处理器消费消息后，依次执行解析、分块、向量化、入库，全部完成后状态改成 success。

为什么要拆成两个接口？因为上传和分块是两个不同粒度的操作。上传只是把文件存起来、记录元数据，很快就能完成。分块涉及解析、向量化等耗时操作，必须异步。而且用户可能上传文档后想先看看文档信息，确认没问题再触发分块。拆开之后，文档更新时也可以只重新执行分块，不需要重新上传文件。

2. 流程关键环节

2.1 文件上传与存储

用户在前端选择文件，点击上传。前端把文件通过 HTTP POST 发送到后端（POST /knowledge-base/{kbId}/docs/upload），后端做这几件事：

1. 校验知识库是否存在
2. 解析文件来源：如果是本地上传，把文件保存到对象存储；如果是 URL 抓取，先从 URL 下载文件再保存
3. 插入记录：在 t_knowledge_document 表插入一条记录，status=pending，记录文件路径、类型、大小、分块配置等
4. 返回响应：返回文档 ID 给前端

注意，上传接口只做文件存储和元数据记录，**不触发分块处理**。上传完成后，文档状态是 pending（待处理），用户可以在前端看到文档列表，确认文件信息无误后再触发分块。

2.2 触发分块

用户在前端点击执行分块按钮，前端调用分块接口（POST /knowledge-base/docs/{docId}/chunk），后端做这几件事：

1. CAS 更新状态：用原子操作把 status 从非 running 改成 running，如果已经是 running，说明有人正在处理，直接拒绝，避免重复触发
2. 如果文档配置了定时刷新，创建或更新定时任务
3. 发送消息：把文档 ID 发送到消息队列，触发异步处理
4. 立即返回：不等处理完成，直接返回成功

为什么用 CAS 更新？因为用户可能手抖点了两次，或者前端重复请求。CAS 保证同一时间只有一个分块任务在执行，不会重复处理。

2.3 异步处理（消息队列）

为什么要异步处理？因为文档处理耗时长。一个 100 页的 PDF，解析要几秒，分块要几秒，向量化要调多次 API，整个流程下来可能要十几秒甚至几分钟。如果同步处理，用户要一直等着，体验很差。

异步处理的好处：

- 不阻塞用户**：触发分块后立即返回，用户可以继续做其他事情
- 支持重试**：处理失败后，消息队列可以自动重试，不需要用户重新操作
- 削峰填谷**：用户集中触发分块时，任务堆积在队列里，慢慢处理，不会压垮服务器

消息队列的选择：RocketMQ、RabbitMQ、Kafka、Redis Stream 都可以。推荐 RocketMQ，因为它支持延迟消息、死信队列、消息轨迹，适合企业级场景。

为了保障大家快速学习项目，项目默认使用 Redis Stream，后续会出教程如何使用 RocketMQ。

2.4 文档解析（Parsing）

异步处理器从消息队列消费到任务后，第一步是解析文件。解析的目标是从原始文件中提取纯文本。

使用 Apache Tika：Tika 是 Apache 的开源项目，支持 1000+ 种文件格式，包括 PDF、Word、Excel、PPT、HTML、Markdown、纯文本等。Tika 的好处是统一接口，你不需要针对每种格式写不同的解析代码，只需要调用 Tika.parseToString(file) 就能拿到纯文本。

支持多种格式：

- PDF**：提取文本、表格、图片等
- Word**：提取正文、标题、列表，忽略样式和批注
- Excel**：把表格转成文本，每行用换行符分隔，每列用制表符分隔
- PPT**：提取每页的标题和正文
- Markdown**：直接读取，保留格式
- HTML**：提取正文，去掉标签和脚本

扫描版 PDF：如果 PDF 是扫描版的（图片），Tika 提取不到文本，需要 OCR 识别。

更新状态：分块接口已经把状态改成了 running，解析失败时更新 status=failed。

2.5 文本分块（Chunking）

解析拿到纯文本后，第二步是分块。分块的目标是把长文本切成适合检索的小段。

系统根据文档的 chunkStrategy 和 chunkConfig 配置执行分块。如果文档没有单独配置，使用知识库的默认配置（通常是递归分块，chunkSize=500、overlapSize=50）。

生成 chunk 列表：分块器输出一个 chunk 列表，每个 chunk 包含：

chunkIndex：	序号，从 0 开始
content：	文本内容
metadata：	元数据，包含文档名、来源、页码等

更新状态：分块失败时，更新 status=failed。

2.6 向量化（Embedding）

分块完成后，第三步是向量化。向量化的目标是把文本 chunk 转成数字向量。

调用 Embedding API：根据知识库的 embedding_model 配置，调用对应的 Embedding API。比如知识库配置的是 qwen-embedding-8b，就调用 SiliconFlow 的 Qwen Embedding API。

批量处理：Embedding API 通常支持批量调用，一次可以处理多个文本。批量调用的好处是减少 HTTP 请求次数，降低延迟。推荐每次处理 10~20 个 chunk，太多会超过 API 的 Token 限制，太少会增加请求次数。

成本控制：Embedding API 按 Token 计费，不同模型和平台价格不同。如果觉得贵，可以用本地 Embedding 模型（Ollama、vLLM），但可能需要 GPU 显卡或服务器。

更新状态：向量化失败时，更新 status=failed。

2.7 入库（Indexing）

向量化完成后，最后一步是入库。入库的目标是把 chunk 和向量写入数据库。

批量插入：不要一个一个插入，要批量插入。PostgreSQL 支持批量 INSERT，一次可以插入几百条记录，性能比逐条插入高几十倍。

事务控制：入库要用事务，要么全部成功，要么全部失败。如果插入到一半失败了，要回滚，不能留下脏数据。

更新文档记录：入库完成后，更新 t_knowledge_document 表：

status=success
chunk_count=N（N 是 chunk 数量）

更新状态：入库成功后，文档处理流程结束。用户可以在前端看到文档状态变成 success，可以开始检索了。如果中间任何环节失败，状态变成 failed。

3. 异常处理与重试

3.1 失败场景

文档处理过程中可能遇到各种失败：

文件损坏： PDF 文件损坏，Tika 解析失败。这种情况无法恢复，只能提示用户重新上传。
不支持的格式： 用户上传了 Tika 不支持的格式，比如加密的 PDF、特殊的 CAD 文件。这种情况也无法恢复，提示用户转换格式后重新上传。
API 调用失败： Embedding API 超时、限流、服务不可用。这种情况可以重试，通常重试几次就能成功。

3.2 重试策略

消息队列支持失败重试，配置重试策略：

重试次数： 最多重试 3 次。如果 3 次都失败，说明问题比较严重，不要无限重试，避免浪费资源。
重试间隔： 指数退避，第 1 次重试间隔 1 秒，第 2 次 2 秒，第 3 次 4 秒。这样可以避免瞬时故障导致的失败。
死信队列： 重试 3 次后仍失败的消息，发送到死信队列。死信队列的消息需要人工介入，排查失败原因，修复后重新处理。
幂等性： 重试时要保证幂等性，避免重复处理。可以在处理前检查文档状态，如果已经是 success，直接跳过。或者先删除已有半成品数据，然后重新开始。

小结

这篇文档讲了企业级知识库系统的宏观设计，从理论到实战的鸿沟、三层架构、核心概念、数据库设计、处理流程、关键难点，把整个系统的骨架搭起来了。

核心要点回顾：

三层架构： 用户层负责交互，业务层负责逻辑，存储层负责持久化。PostgreSQL + pgvector 替代 Milvus，单体架构，事务一致性更好。
三大核心概念： 知识库是文档的容器，文档是知识库的基本单元，分块是检索的基本单元。它们是一对多的关系，通过外键关联。
数据库表设计： t_knowledge_base、t_knowledge_document、t_knowledge_chunk 三张核心表，字段设计要考虑业务需求、性能优化、扩展性。

文档处理流程：上传 → 解析 → 分块 → 向量化 → 入库，异步处理，状态流转，异常重试。消息队列解耦各个环节，支持并发和重试。

这些是宏观设计，后续的实战篇会深入每个环节的具体实现。

